

Ht no:2303A51827

Batch:26

Task 1: Auto-Generating Function Documentation in a Shared Codebase

Scenario

You have joined a development team where several utility functions are already implemented, but the code lacks proper documentation. New team members are struggling to understand how these functions should be used.

Task Description

You are given a Python script containing multiple functions without any docstrings.

Using an AI-assisted coding tool:

- Ask the AI to automatically generate Google-style function docstrings for each function

- Each docstring should include:

- o A brief description of the function

- o Parameters with data types

- o Return values

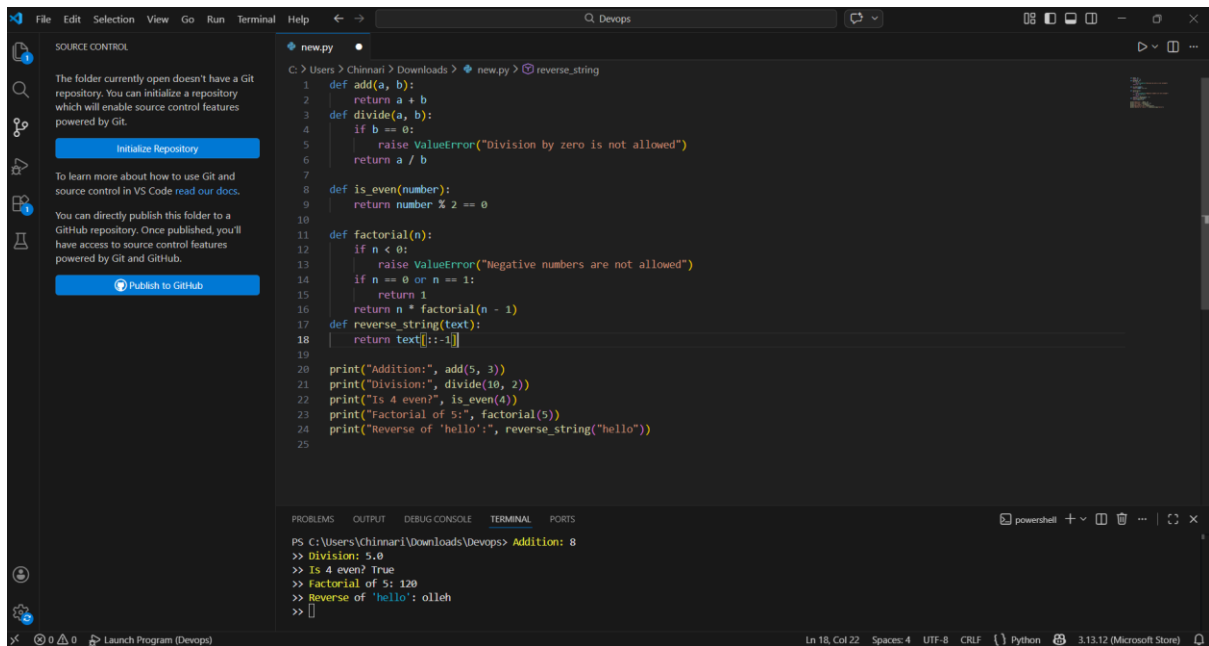
- o At least one example usage (if applicable)

Experiment with different prompting styles (zero-shot or context-based) to observe quality differences.

Expected Outcome

- A Python script with well-structured Google-style docstrings
- Docstrings that clearly explain function behavior and usage

Improved readability and usability of the codebase give code for ths question for vs code



Task 2: Enhancing Readability Through AI-Generated Inline

Comments

Scenario

A Python program contains complex logic that works correctly but is difficult to understand at first glance. Future maintainers may find it hard to debug or extend this code.

Task Description

You are provided with a Python script containing:

- Loops
- Conditional logic
- Algorithms (such as Fibonacci sequence, sorting, or searching)

Use AI assistance to:

- Automatically insert inline comments only for complex or non-obvious logic
- Avoid commenting on trivial or self-explanatory syntax

The goal is to improve clarity without cluttering the code.

Expected Outcome

- A Python script with concise, meaningful inline comments
- Comments that explain why the logic exists, not what Python syntax does

- Noticeable improvement in code readability i want code for this with output

```

1  def fibonacci(n):
2      sequence = []
3      a, b = 0, 1
4
5      for _ in range(n):
6          sequence.append(a)
7          a, b = b, a + b
8
9      return sequence
10
11 def bubble_sort(arr):
12     n = len(arr)
13
14     for i in range(n):
15         for j in range(0, n - i - 1):
16             if arr[j] > arr[j + 1]:
17                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
18
19     return arr
20
21 def binary_search(arr, target):
22     left, right = 0, len(arr) - 1
23
24     while left <= right:
25         mid = (left + right) // 2
26         if arr[mid] == target:
27             return mid
28         elif arr[mid] < target:
29             left = mid + 1
30
31     return -1
32
33 if __name__ == "__main__":
34     print("Fibonacci (first 7 numbers):")
35     print(fibonacci(7))
36
37     print("\nBubble Sort:")
38     numbers = [64, 34, 25, 12, 22, 11, 90]
39     print("Before sorting:", numbers)
40     sorted_numbers = bubble_sort(numbers.copy())
41     print("After sorting:", sorted_numbers)
42
43     print("\nBinary Search:")
44     index = binary_search(sorted_numbers, 25)
45     print("Element 25 found at index:", index)
46

```

Output:

```

Fibonacci (first 7 numbers):
[0, 1, 1, 2, 3, 5, 8]

Bubble Sort:
Before sorting: [64, 34, 25, 12, 22, 11, 90]

```

```

19  def binary_search(arr, target):
20
21      while left <= right:
22          mid = (left + right) // 2
23          if arr[mid] == target:
24              return mid
25          elif arr[mid] < target:
26              left = mid + 1
27
28      else:
29          right = mid - 1
30
31      return -1
32
33 if __name__ == "__main__":
34     print("Fibonacci (first 7 numbers):")
35     print(fibonacci(7))
36
37     print("\nBubble Sort:")
38     numbers = [64, 34, 25, 12, 22, 11, 90]
39     print("Before sorting:", numbers)
40     sorted_numbers = bubble_sort(numbers.copy())
41     print("After sorting:", sorted_numbers)
42
43     print("\nBinary Search:")
44     index = binary_search(sorted_numbers, 25)
45     print("Element 25 found at index:", index)
46

```

Output:

```

After sorting: [11, 12, 22, 25, 34, 64, 90]

Binary Search:
Element 25 found at index: 3

```

Task 3: Generating Module-Level Documentation for a Python

Package

Scenario

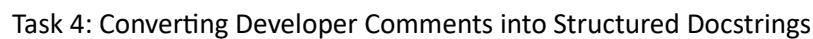
Your team is preparing a Python module to be shared internally (or uploaded to a repository). Anyone opening the file should immediately understand its purpose and structure.

Task Description

- The purpose of the module
- Required libraries or dependencies
- A brief description of key functions and classes
- A short example of how the module can be used

Expected Outcome

- A well-written multi-line module-level docstring
- Clear overview of what the module does and how to use it
- Documentation suitable for real-world projects or repositories



In a legacy project, developers have written long explanatory comments inside functions instead of proper docstrings. The team now wants to standardize documentation.

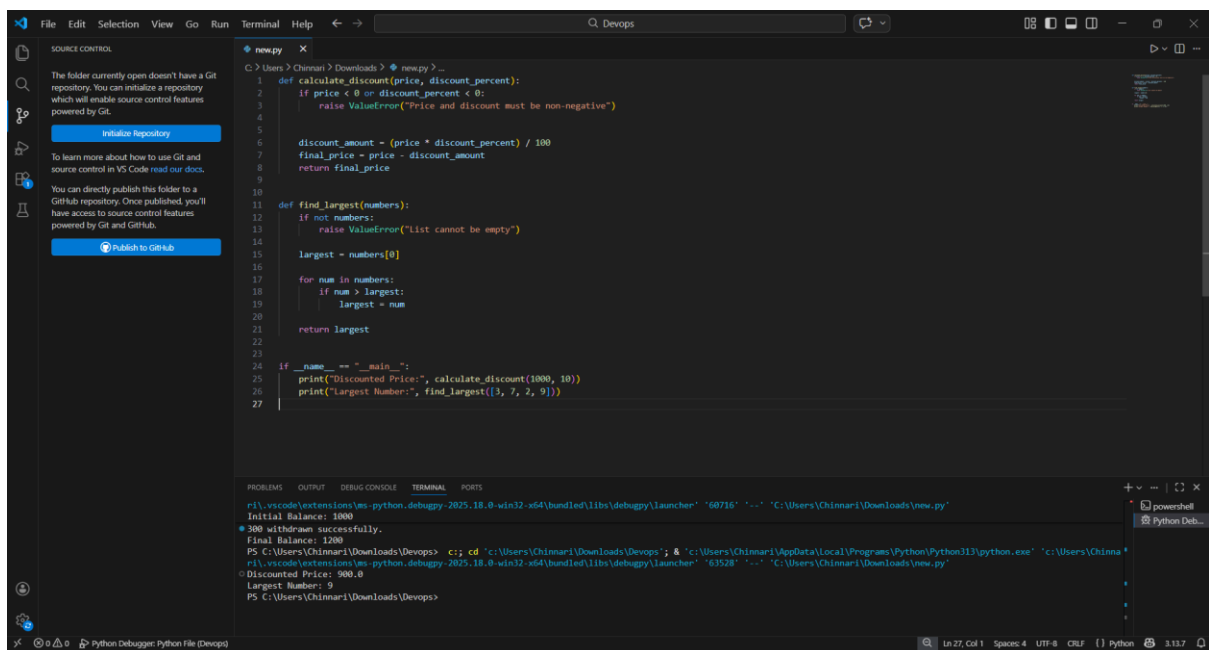
You are given a Python script where functions contain detailed inline comments explaining their logic.

Use AI to:

- Automatically convert these comments into structured Google-style or NumPy-style docstrings
- Preserve the original meaning and intent of the comments
- Remove redundant inline comments after conversion

Expected Outcome

- Functions with clean, standardized docstrings
- Reduced clutter inside function bodies
- Improved consistency across the codebase i want code for this with output



```
File Edit Selection View Go Run Terminal Help
new.py
1 def calculate_discount(price, discount_percent):
2     if price < 0 or discount_percent < 0:
3         raise ValueError("Price and discount must be non-negative")
4
5     discount_amount = (price * discount_percent) / 100
6     final_price = price - discount_amount
7     return final_price
8
9
10
11 def find_largest(numbers):
12     if not numbers:
13         raise ValueError("List cannot be empty")
14
15     largest = numbers[0]
16
17     for num in numbers:
18         if num > largest:
19             largest = num
20
21     return largest
22
23
24 if __name__ == "__main__":
25     print("Discounted Price:", calculate_discount(1000, 10))
26     print("Largest Number:", find_largest([3, 7, 2, 9]))
27
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
r:\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bin\debugpy_launcher "68716" "..." "C:\Users\Chinnar\Downloads\new.py"
Initial Balance: 1000
Final Balance: 1200
PS C:\Users\Chinnar\Downloads\Devops> c:; cd 'C:\Users\Chinnar\Downloads\Devops'; & 'c:\Users\Chinnar\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Chinnar\Downloads\new.py'
Discounted Price: 900.0
Largest Number: 9
PS C:\Users\Chinnar\Downloads\Devops>
```

Task 5: Building a Mini Automatic Documentation Generator

Scenario

Your team wants a simple internal tool that helps developers start documenting new Python files quickly, without writing documentation from scratch.

Task Description

Design a small Python utility that:

- Reads a given .py file
- Automatically detects:
 - o Functions
 - o Classes

- Inserts placeholder Google-style docstrings for each detected function or class

AI tools may be used to assist in generating or refining this utility.

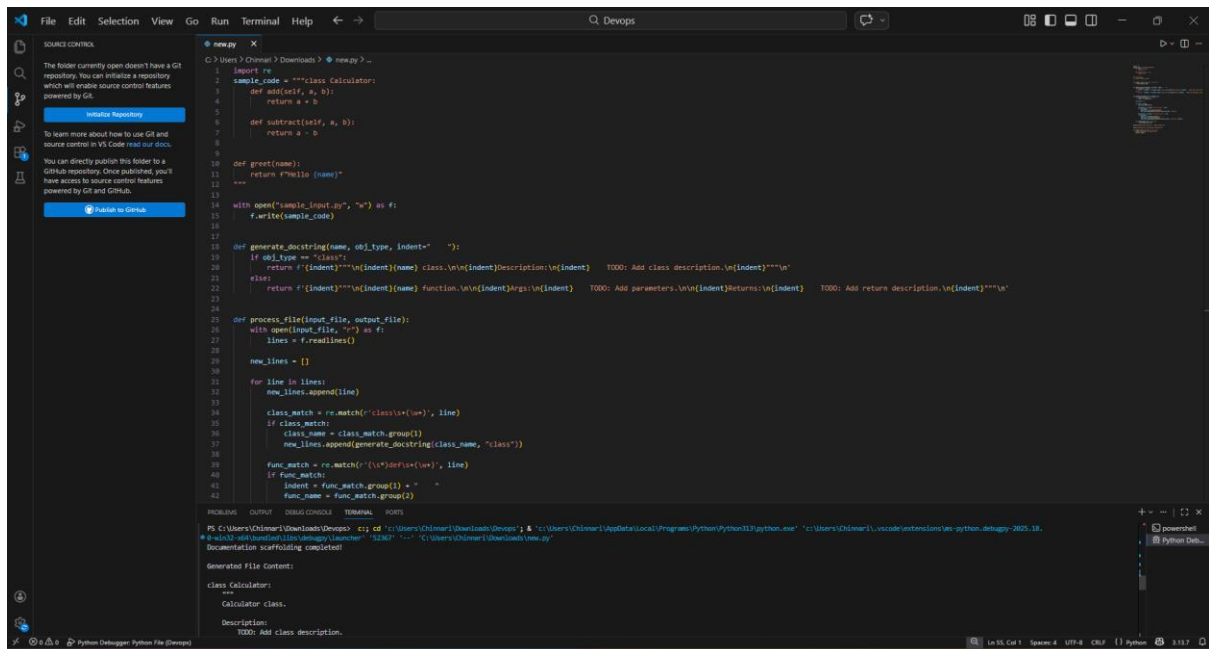
Note: The goal is documentation scaffolding, not perfect

documentation.

Expected Outcome

- A working Python script that processes another .py file
- Automatically inserted placeholder docstrings
- Clear demonstration of how AI can assist in documentation

automation i want code for this with output



The screenshot shows a Visual Studio Code editor window with a file named 'new.py' open. The file contains a Python script that processes another Python file ('sample.py') and generates documentation. The script includes functions for parsing class and function definitions and inserting docstrings. The terminal at the bottom shows the command to run the script and the resulting output, which is a formatted Python class definition for a 'Calculator' class with docstrings.

```
1 import re
2 sample_code = """class Calculator:
3     def add(self, a, b):
4         return a + b
5
6     def subtract(self, a, b):
7         return a - b
8
9
10    def greet(name):
11        return f"Hello {name}"
12    """
13
14    with open("sample_input.py", "r") as f:
15        f.write(sample_code)
16
17
18    def generate_docstring(name, obj_type, indent="    "):
19        if obj_type == "class":
20            return f'({indent})"""({name}) class.\n\n({indent})Description:\n\n({indent})    TODO: Add class description.\n\n({indent})"""'
21        else:
22            return f'({indent})"""({name}) function.\n\n({indent})Args:\n\n({indent})    TODO: Add parameters.\n\n({indent})Returns:\n\n({indent})    TODO: Add return description.\n\n({indent})"""'
23
24
25    def process_file(input_file, output_file):
26        with open(input_file, "r") as f:
27            lines = f.readlines()
28
29        new_lines = []
30
31        for line in lines:
32            new_lines.append(line)
33
34            class_match = re.match(r'^(class\s+)(\w+)', line)
35            if class_match:
36                class_name = class_match.group(1)
37                new_lines.append(generate_docstring(class_name, "class"))
38
39            func_match = re.match(r'^(def\s+)(\w+)', line)
40            if func_match:
41                func_name = func_match.group(1)
42                new_lines.append(generate_docstring(func_name, "function"))
```

Terminal Output:

```
PS C:\Users\Chinmay\Downloads\Devops> cd .\c:\Users\Chinmay\Downloads\Devops & & "c:\Users\Chinmay\AppData\Local\Programs\Python\Python311\python.exe" "c:\Users\Chinmay\Downloads\new.py"
Documentation scaffolding completed
Generated File Content:
class Calculator:
    """
    Calculator class.

    Description:
    TODO: Add class description.
    """
```